

What happens when load factor increases?

The load factor ($a = n/m$) represents the mathematically expected number of artworks per room. As we scaled our testing from the small debug dataset ($a = 2$) to the large performance dataset ($a = 20$), the expected baseline load shifted upward proportionally.

Our designed universal hash function successfully maintained a maximum load of 22 on the large dataset, which is very close to the theoretical average of 20. This demonstrates the algorithm's ability to distribute data and scale reliably. However, when the load factor increases to the point that it significantly exceeds the physical room capacity ($B = 6$), overflows become mathematically unavoidable. In our large dataset experiment, every single one of the 10 rooms overflowed. This highlights a fundamental physical constraint: while a robust hash function can balance the load evenly across all available bins, it cannot circumvent physical capacity limitations. If an exhibition is heavily overbooked, the physical space must simply be increased.

What failure cases did you observe?

During our testing, we observed a highly counterintuitive failure case related to algorithmic vulnerability and predictable inputs. In our small dataset test ($n=6$, $m=3$), our designed hash resulted in a maximum load of 3 (causing one overflow), while a weak hash effectively functioning as $id \% m$ distributed the items perfectly with a max load of exactly 2. This same "perfect illusion" occurred in the large dataset, where the weak hash achieved a mathematically perfect max load of 20, slightly outperforming our designed hash's max load of 22.

The failure case here is **clustering due to predictable data**. The weak hash only achieved a perfect distribution because our test dataset consisted of perfectly sequential integer IDs (1 through 200). If this were applied in a real-world scenario where artwork IDs followed a non-sequential pattern—such as a collection cataloged in increments of ten (10, 20, 30...)—this simple modulo-based hash would systematically assign every single artwork to Room 0. This would cause catastrophic clustering, leaving the rest of the gallery entirely empty.

The Impact of Tuning Constants (a, b, and m):

To further evaluate our design and prevent the failure cases mentioned above, we tested various constants to observe their impact on the layout distribution:

- **Tuning the Multipliers (a and b):** We compared different mathematical multipliers: a weak pair ($a=1$, $b=0$), a standard pair ($a=31$, $b=17$), and our primary golden-ratio multiplier ($a=2654435761$). On our sequential dataset, all three yielded nearly identical maximum loads ranging from 20 to 22. While tuning a and b is mathematically critical to scramble bits and prevent collisions when processing unpredictable real-world data, our controlled sequential dataset masked this benefit. The slight variance introduced by our primary designed hash (max load 22) is a necessary and acceptable trade-off for the algorithmic robustness required to prevent clustering.
- **Tuning the Modulo (m):** Altering the modulo constant yielded a drastic and immediate physical impact on the results. When we doubled the number of exhibition rooms from $m=10$ to $m=20$, the load factor (a) halved from 20 down to 10. Consequently, the empirical maximum load dropped proportionally from 22 down to 12.

Ultimately, this testing confirms a core property of hashing in layout design: while tuning the multipliers (a and b) ensures a safe, even distribution by preventing structural clustering, scaling the number of bins (m) is the only definitive way to reduce the absolute maximum load on any individual room.